AAE4203 Guidance and Navigation

GNSS Data Collection, Single Point Positioning and an Accuracy Challenge

Laboratory Tutorial · Week 9 · Semester 1, 2026/27

Dr Weisong Wen and the AAE4203 teaching team

Department of Aeronautical and Aviation Engineering
The Hong Kong Polytechnic University
welson.wen@polyu.edu.hk · Room R820

Laboratory session: 30 October 2026 · Version 3.0

Abstract*—This tutorial guides student groups through a complete GNSS positioning experiment. Each group collects 15–20 minutes of static raw data with a u-blox ZED-F9P receiver and u-center near Block X, converts and processes the data with RTKLIB, implements a weighted least-squares single point positioning (SPP) solver in Python, and validates the solution against independent references. Groups then improve the accuracy of their solver on an urban benchmark dataset; they submit the estimated trajectory as a CSV file and are ranked on a class leaderboard by horizontal root-mean-square error. Generative AI tools may be used for coding, subject to the rules in Section 3. The theory sections give fully consistent worked examples for position, clock bias, velocity and clock drift.*

Keywords*—GNSS, single point positioning, weighted least squares, RTKLIB, u-blox, multi-constellation, multipath, leaderboard.*

Contents

1 Introduction 4
2 Laboratory Organisation in 2026/27 4
2.1 Overview of the workflow 4
2.2 Datasets provided 5
2.3 Session timeline 5
2.4 Groups 6
3 Requirements, Assessment and Rules 7
3.1 Deliverables 7
3.2 Lab report 7
3.3 Use of generative AI tools 8
3.4 Contacts 8
4 Theory of Single Point Positioning 8
4.1 Pseudorange measurement model 8
4.2 Linearisation and weighted least squares 8
4.3 Extension to several constellations 9
4.4 Worked example: position and clock bias 9
4.5 Doppler measurements, velocity and clock drift 10
4.6 Error budget and expected accuracy 11
5 Field Data Collection with a u-blox Receiver 11
5.1 Receiver kit 12
5.2 Logging raw data with u-center 12
5.3 Site and procedure 14
6 Processing with RTKLIB 15
6.1 Conversion to RINEX with RTKCONV 16
6.2 Reference-station data from HK SatRef 16
6.3 Post-processing with RTKPOST 18
7 Conversion of Raw Data to CSV 21
7.1 RINEX to CSV with rinex2csv.m 21
7.2 Receiver solution from UBX with pyubx2 23
8 SPP Implementation and Accuracy Improvement 23
8.1 Baseline implementation 23
8.2 Ideas for improving accuracy 23
8.3 Working with AI tools 23
8.4 Verification on the example dataset 24
8.5 Validating your own data without ground truth 24
8.6 Example results on the Block X dataset 25

9 Accuracy Challenge: Submission and Leaderboard 27
9.1 CSV file format 27
9.2 Upload 27
9.3 Evaluation metrics 28
9.4 Ranking rules 28
10 Checklist and Concluding Remarks 28
References 28

1 Introduction

Global Navigation Satellite Systems (GNSS) provide position, velocity and time to users anywhere on or near the Earth. In dense cities such as Hong Kong, however, buildings block and reflect the satellite signals, so the accuracy of a standalone receiver can degrade from a few metres to tens of metres. This laboratory takes you from raw measurements to your own positioning solution and asks you to make that solution as accurate as possible.

After completing the laboratory you will be able to:

1. collect raw GNSS measurements with a u-blox receiver and the u-center software;
2. convert raw UBX data to RINEX and CSV and post-process it with RTKLIB (RTKCONV, RTKPOST, RTKPLOT);
3. implement weighted least-squares SPP in Python and extend it to several constellations;
4. improve positioning accuracy in an urban environment (weighting, masking, outlier and multipath mitigation) and validate a solution without ground truth by comparison with independent references;
5. quantify the improvement against the reference trajectory of an urban benchmark;
6. use generative AI tools responsibly to write and debug navigation code, and document that use.



Fig. 1. Two receiver architectures used in this laboratory: (left) a module-based GNSS receiver connected to a laptop; (right) an integrated GNSS positioning unit.

What is new in 2026/27. Compared with last year, three elements have been added: (i) a challenging urban benchmark dataset from UrbanNav [4], (ii) a CSV submission and a class leaderboard ranked by accuracy, and (iii) explicit permission to use AI tools for coding, together with rules for doing so. Self-collected data remain central to the report; because no ground truth is available for them, they are validated by comparison with independent solutions (Section 8.5) and are not ranked.

2 Laboratory Organisation in 2026/27

2.1 Overview of the workflow

Fig. 2 summarises the laboratory. Three datasets enter the same processing chain, and each answers a different question. Your own field data, collected during the session, answer the question “can we collect and process real measurements?”; they have no ground truth and are validated by comparison. The example dataset (Block X, recorded in 2025) answers “does our code give the right answer?”, because its reference results are known; use it as a warm-up before the session and as a regression test after every change to your code. The benchmark dataset answers “how accurate is our method?”: you submit your estimated trajectory, and the teaching team compares it with the reference ground truth to evaluate positioning accuracy across groups.

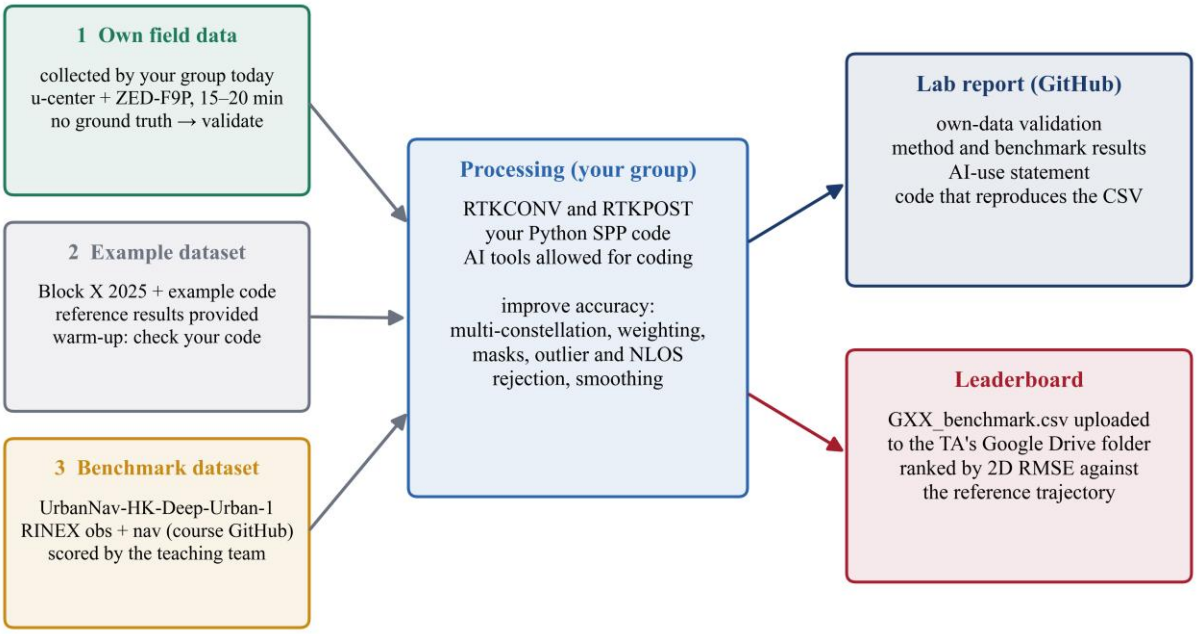


Fig. 2. Workflow of the 2026/27 laboratory: three datasets, one processing chain, and two outputs (leaderboard submission and lab report).

2.2 Datasets provided

Table 1. Datasets used in the laboratory.

Dataset	Content	Ground truth	Purpose
Own field data	UBX log recorded by your group (15–20 min, static) near Block X	none	data collection, own-data validation and discussion in the report
Example (Block X, 2025)	UBX log, RINEX .obs/.nav, CSV files from rinex2csv.m, receiver solution (NAV-HPPOSECEF), example Python code	reference results provided	warm-up and regression test: check that your code reproduces the example results
UrbanNav Benchmark	UrbanNav-HK-Deep-Urban-1 RINEX observation file and matching multi-constellation navigation file [4]	reference trajectory kept by the teaching team for scoring	accuracy challenge and leaderboard (Section 9)

The benchmark files urbannav_deep1.obs (u-blox ZED-F9P observations of UrbanNav-HK-Deep-Urban-1) and urbannav_deep1.nav (matching multi-constellation broadcast navigation data) are in the folder Lab1/benchmark of the course GitHub repository (Section 5). The reference trajectory used for scoring is not included. UrbanNav is a public dataset [4] and its ground truth is published; using that ground truth in any way (as a submission, for tuning or for checking) is not allowed (Section 3.3). The example dataset comes with the receiver's own high-precision solution, which serves as a reference for debugging; it is not a survey-grade ground truth.

Why keep an example dataset when an UrbanNav benchmark exists? The example dataset provides reference results for checking a baseline implementation before testing on the more challenging UrbanNav data.

2.3 Session timeline

The session lasts three hours. Ten receiver kits are available for 20 groups, so the field collection runs in two batches (Fig. 3). Kits K01–K10 are used first by groups G01–G10 (batch 1) and then handed over to the partner groups G11–G20 (batch 2): kit Kn goes from group Gn to group G(n+10), for example K01 from

G01 to G11. While batch 1 is in the field, batch 2 checks its software and reproduces the results of the example dataset. After the field collection, every group follows the same order: process and validate its own data, then work on the benchmark challenge. Batch 1 starts its benchmark block with the example check that batch 2 did at the beginning. Both batches can keep improving their benchmark solution after the session; the final ranking uses the files present at the deadline, so the two batches have the same total time.

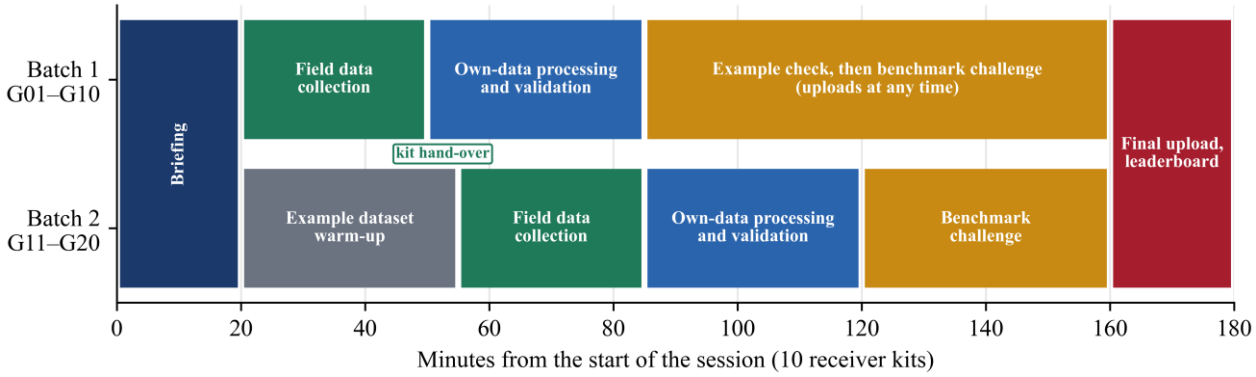


Fig. 3. Timeline of the three-hour laboratory session with 10 receiver kits shared by two batches.

Table 2. Detailed schedule of the session.

Time from start	Batch 1 (G01–G10)	Batch 2 (G11–G20)
before the session	install u-center, RTKLIB and Python; run the example code	same as batch 1
0:00–0:20	briefing; pick up kits K01–K10	briefing
0:20–0:50	**field data collection near Block X (15–20 min static)**	software check; reproduce the example results (Section 8.6)
0:50–0:55	hand the kit over to the partner group	receive the kit from the partner group
0:55–1:25	own data: RTKCONV, RTKPOST, rinex2csv, your SPP; validation (Section 8.5)	**field data collection near Block X (15–20 min static)**
1:25–2:00	example check; benchmark challenge	return the kit; own data: processing and validation (Section 8.5)
2:00–2:40	benchmark challenge, uploads at any time	benchmark challenge, uploads at any time
2:40–3:00	final uploads; preliminary leaderboard	final uploads; preliminary leaderboard

2.4 Groups

The 78 students work in 20 groups of three or four, the same groups as for the group presentation. Each group collects one dataset with one receiver kit and submits one report and one leaderboard entry. Every member should be able to explain the whole workflow. A suggested division of roles is: hardware set-up and logging; data conversion and RTKLIB processing; SPP coding and accuracy improvement; analysis, report and repository.

Table 3. Groups for the 2026/27 laboratory (identical to the group-presentation groups), with the receiver kit used by each group; the left column is batch 1 and the right column batch 2. Member names and student numbers are removed from this public version; the full list is on Blackboard.

Group	Group size	Group	Group size
G01 K01	4 members (names on Blackboard)	G11 K01	4 members (names on Blackboard)
G02 K02	4 members (names on Blackboard)	G12 K02	4 members (names on Blackboard)
G03 K03	4 members (names on Blackboard)	G13 K03	4 members (names on Blackboard)
G04 K04	3 members (names on Blackboard)	G14 K04	4 members (names on Blackboard)

Group	Group size	Group	Group size
G05 K05	4 members (names on Blackboard)	G15 K05	4 members (names on Blackboard)
G06 K06	4 members (names on Blackboard)	G16 K06	4 members (names on Blackboard)
G07 K07	4 members (names on Blackboard)	G17 K07	4 members (names on Blackboard)
G08 K08	4 members (names on Blackboard)	G18 K08	4 members (names on Blackboard)
G09 K09	3 members (names on Blackboard)	G19 K09	4 members (names on Blackboard)
G10 K10	4 members (names on Blackboard)	G20 K10	4 members (names on Blackboard)

3 Requirements, Assessment and Rules

3.1 Deliverables

Table 4. Deliverables of each group.

Deliverable	Content	Where and when
Leaderboard entry	GXX_benchmark.csv: your trajectory for the benchmark dataset (format in Section 9)	TA's Google Drive folder; preliminary leaderboard at the end of the session
Own-data solution	GXX_owndata.csv: your trajectory for your own data (same format)	same folder (not ranked)
Lab report	PDF, at most 10 pages excluding the cover; structure in Table 5	Blackboard; deadline announced on Blackboard
Code and data	GitHub repository: raw data, scripts, README.md; it must reproduce both CSV files	link given in the report

3.2 Lab report

Use the report template on Blackboard (PolyU_AAE4203_Lab_report_template.docx). The report must not exceed 10 pages excluding the cover page, and code listings in the report are limited to 2 pages; the complete, documented code belongs in the repository. Table 5 lists the required parts.

Table 5. Structure of the lab report.

Part	Required content
1. Data collection	Hardware and software set-up, site photos, time, environment and sky view near Block X.
2. Raw data analysis	Satellites tracked per constellation, C/N_0, elevation and sky plot, DOP, other quality indicators.
3. SPP algorithm	Measurement model, linearisation and weighted least squares; your Python implementation.
4. Own-data validation	Your SPP compared with RTKPOST and with the receiver solution (mean and standard deviation of the E/N/U differences), scatter about the mean, input checks (Section 8.5).
5. Benchmark results	Each technique you tried, why, and its measured effect (ablation table); final 2D/3D RMSE, 95th percentile, availability and rank; discussion of urban effects.
6. AI-use statement	Tools used, what they were used for, key prompts (appendix), and how the group verified the output.

3.3 Use of generative AI tools

You may use AI assistants (for example ChatGPT, Claude, Gemini, DeepSeek or Copilot) during the session and afterwards to write, explain and debug code. The aim of the challenge is to obtain the most accurate trajectory with methods that you understand. The following rules apply.

Rules for AI-assisted work

- Every group member must be able to explain the algorithm and the code that produced the submitted CSV.
- Verify AI-generated code on the example dataset before applying it to the benchmark (Section 8.4).
- State in the report which tools you used and for what, and list the key prompts in an appendix.
- The submitted trajectory must be computed by your code from the provided observations. Submitting another group's results, a receiver solution, the published UrbanNav ground truth or any position not computed by your own code leads to disqualification from the leaderboard and is handled under PolyU's academic integrity regulations.
- Do not upload other groups' data or personal data to AI services.

3.4 Contacts

Lecturer: Dr Weisong WEN, Department of Aeronautical and Aviation Engineering, e-mail welson.wen@polyu.edu.hk, office R820. The names and e-mail addresses of the teaching assistants for 2026/27 are posted on Blackboard. Please contact the TAs first for technical problems with the receiver kits, software or submission.

4 Theory of Single Point Positioning

This section summarises the models taught in Weeks 4–7 and gives worked examples whose numbers are mutually consistent and can be reproduced with a few lines of Python. Vectors are bold, ECEF coordinates are used for positions, and all clock terms are expressed in metres.

4.1 Pseudorange measurement model

For satellite i the pseudorange measured by the receiver is

$$\rho_i = \|p^i - p\| + c\,\delta t_r - c\,\delta t^i + I_i + T_i + \varepsilon_i \tag{1}$$

where $p^i = [x^i, y^i, z^i]^T$ is the satellite position at transmission time, $p = [x, y, z]^T$ is the receiver position, c is the speed of light (299 792 458 m/s), δt_r and δt^i are the receiver and satellite clock offsets, I_i and T_i are the ionospheric and tropospheric delays, and ε_i collects multipath, non-line-of-sight (NLOS) errors and noise. The satellite clock offset is computed from the broadcast ephemeris; I_i and T_i are predicted with the Klobuchar and Saastamoinen models. Writing $b = c\,\delta t_r$ for the receiver clock bias in metres, the corrected pseudorange is

$$\tilde{\rho}_i = \rho_i + c\,\delta t^i - I_i - T_i = \|p^i - p\| + b + \varepsilon_i. \tag{2}$$

4.2 Linearisation and weighted least squares

Let $x = [x, y, z, b]^T$ and let x_0 be the current estimate. With the geometric range $r_i^0 = \|p^i - p_0\|$ and the unit line-of-sight vector $e_i = (p^i - p_0)/r_i^0$ from the receiver to the satellite, a first-order Taylor expansion gives

$$y_i = \tilde{\rho}_i - r_i^0 - b_0 \approx -e_i^T\,\Delta p + \Delta b + \varepsilon_i, \tag{3}$$

so that for n satellites $y = H\,\Delta x + \boldsymbol{\varepsilon}$ with

$$H = \begin{bmatrix} -e_1^T & 1 \\ \vdots & \vdots \\ -e_n^T & 1 \end{bmatrix}, \qquad W = \mathrm{diag}(w_1, \ldots, w_n), \qquad w_i = \sin^2\theta_i, \tag{4}$$

where θ_i is the elevation angle; low satellites are more affected by multipath and the atmosphere and therefore receive smaller weights. The weighted least-squares (WLS) correction and the update are

$$\Delta\hat{x} = (H^T W H)^{-1} H^T W\, y, \qquad x_{k+1} = x_k + \Delta\hat{x}, \tag{5}$$

and the iteration (Gauss–Newton) stops when $\|\Delta\hat{p}\|$ falls below a threshold such as 1 mm. If the weights equal $w_i = \sigma_0^2/\sigma_i^2$, the covariance of the estimate is $C_{\hat{x}} = \sigma_0^2 (H^T W H)^{-1}$. With $W = I$, $Q = (H^T H)^{-1}$ defines the dilution of precision, for example GDOP $= \sqrt{\mathrm{tr}\, Q}$; HDOP and VDOP follow after rotating the position block of Q to East–North–Up (ENU) coordinates.

4.3 Extension to several constellations

GPS, BeiDou (BDS), Galileo and GLONASS keep their own system times, so a receiver that combines them sees a different clock bias for each system. With one bias per system, e.g. b_G, b_C and b_E, the state becomes $x = [x, y, z, b_G, b_C, b_E]^T$ and a BDS satellite contributes the row

$$h_i^T = [-e_i^T \quad 0 \quad 1 \quad 0]. \tag{6}$$

At least $3 + m$ satellites are needed for m systems, but in an urban canyon the extra satellites usually improve the geometry far more than the extra unknowns cost. Two practical points: BDS broadcast ephemerides refer to BDS time (BDT = GPST − 14 s), and GLONASS satellites use different frequencies, so start with GPS + BDS + Galileo.

4.4 Worked example: position and clock bias

A receiver near PolyU observes five GPS satellites. The pseudoranges below are already corrected for the satellite clock offsets; the ionospheric and tropospheric delays predicted by the models are listed separately. The initial estimate is $x_0 = [-2418000.0, 5385000.0, 2406000.0, 0]^T$ m, about 1 km from the true position.

Table 6. Input data of the position example (epoch on the lab day).

PRN	Az. (°)	El. (°)	x^i (m)	y^i (m)	z^i (m)	ρ_i (m)	I_i (m)	T_i (m)	w_i
G05	35	72	−12279188.8	18509172.8	14562473.9	20431181.26	4.85	2.42	0.9045
G13	140	48	−18600547.8	18880989.3	−1720890.7	21475775.35	5.60	3.09	0.5523
G15	230	31	4805112.5	25690114.1	−4728903.9	22705642.17	7.12	4.47	0.2653
G20	310	55	697304.3	21069860.0	16155753.3	21093838.34	5.21	2.81	0.6710
G29	85	22	−25267228.8	3267032.6	7505148.0	23511370.83	8.94	6.14	0.1403

Step 1 — geometric ranges at x_0. For G05,

$$r_1^0 = \sqrt{(-9861188.8)^2 + (13124172.8)^2 + (12156473.9)^2} = 20427109.78 \text{ m} \tag{7}$$

and the five ranges are 20427109.78, 21472023.83, 22701935.30, 21089705.31, 23506900.11 m.

Step 2 — observation vector, $y_i = \rho_i - I_i - T_i - r_i^0 - b_0$:

$$y = \begin{bmatrix} 4064.21 \\ 3742.83 \\ 3695.28 \\ 4125.01 \\ 4455.64 \end{bmatrix} \text{m} \tag{8}$$

All five values are close to 4 km because $b_0 = 0$ while the true receiver clock bias is about 4.5 km (15 μs); the differences between them carry the 1 km position error.

Step 3 — design matrix (rows $[-e_i^T, 1]$) and weights:

$$H = \begin{bmatrix} 0.4828 & -0.6425 & -0.5951 & 1.0000 \\ 0.7537 & -0.6285 & 0.1922 & 1.0000 \\ -0.3182 & -0.8944 & 0.3143 & 1.0000 \\ -0.1477 & -0.7437 & -0.6520 & 1.0000 \\ 0.9720 & 0.0901 & -0.2169 & 1.0000 \end{bmatrix} \tag{9}$$

$$W = \mathrm{diag}(0.9045, 0.5523, 0.2653, 0.6710, 0.1403) \tag{10}$$

Step 4 — WLS correction from Eq. (5) and updated state:

$$\Delta\hat{x} = \begin{bmatrix} -204.78 \\ 852.70 \\ -350.51 \\ 4501.46 \end{bmatrix} \text{ m}, \qquad x_1 = \begin{bmatrix} -2418204.78 \\ 5385852.70 \\ 2405649.49 \\ 4501.46 \end{bmatrix} \text{ m} \tag{11}$$

Table 7. Gauss–Newton iterations of the position example.

Iteration k	Δx (m)	Δy (m)	Δz (m)	Δb (m)	$\|\|\Delta p\|\|$ (m)
1	−204.776	852.702	−350.512	4501.456	944.4011
2	0.002	−0.010	0.012	−0.017	0.0154
3	0.000	0.000	0.000	0.000	0.0000

Step 5 — convergence. The second correction is about 1.5 cm and the third is below 0.1 mm, giving

$$\hat{p} = \begin{bmatrix} -2418204.774 \\ 5385852.693 \\ 2405649.499 \end{bmatrix} \text{ m}, \qquad \hat{b} = 4501.439 \text{ m} \quad (\delta t_r = 15.015\ \mu\text{s}). \tag{12}$$

The post-fit residuals are 0.87, −0.97, 1.51, −1.04, 0.36 m. Because the measurements contain multipath and noise of up to 3 m, the estimate is not exact: compared with the true position the error is −0.12 m East, −1.48 m North and 5.46 m Up, i.e. 1.49 m horizontally and 5.66 m in 3D. The geometry gives GDOP = 3.42, HDOP = 1.54 and VDOP = 2.47, which is why the vertical error is the largest.

4.5 Doppler measurements, velocity and clock drift

The Doppler shift $f_{D,i}$ measured on L1 ($f_{L1} = 1575.42$ MHz, $\lambda = c/f_{L1} = 0.190294$ m) gives the range rate $\dot{\rho}_i = -\lambda f_{D,i}$ (an approaching satellite has a positive Doppler shift). With the satellite velocity v^i, the receiver velocity v and the receiver clock drift $d = c\,\dot{\delta t}_r$ in m/s,

$$\dot{\rho}_i = (v^i - v)^T e_i + d + \dot{\varepsilon}_i, \qquad y_i = \dot{\rho}_i - (v^i)^T e_i = -e_i^T v + d + \dot{\varepsilon}_i. \tag{13}$$

The model is linear in $[v^T, d]^T$ and uses the same design matrix H as the position solution, so a single WLS step gives the answer. As an example, consider the same five satellites, evaluated at $\hat{p}$:

Table 8. Doppler example: measurements, satellite velocities and observations.

PRN	f_D (Hz)	$\dot{\rho}_i$ (m/s)	v_x^i (m/s)	v_y^i (m/s)	v_z^i (m/s)	y_i (m/s)
G05	1182.41	−225.005	−114.85	2347.03	−3079.96	44.700
G13	−3454.61	657.390	−1870.62	−2086.63	−2674.81	44.973
G15	506.20	−96.327	2209.15	171.74	3177.74	45.839
G20	−981.17	186.710	−850.59	−2281.98	3012.80	45.136
G29	3025.88	−575.806	−73.27	3455.08	−1750.69	44.191

For G05, $\dot{\rho}_1 = -0.190294 \times 1182.41 = -225.005$ m/s. Solving the WLS problem gives

$$\hat{v} = \begin{bmatrix} -0.624 \\ -0.568 \\ 0.552 \end{bmatrix} \text{ m/s (ECEF)}, \qquad \hat{d} = 44.972 \text{ m/s} \quad (\hat{\delta t}_r = 150.0 \text{ ns/s}). \tag{14}$$

Rotated to ENU with $v_{ENU} = R\, v_{ECEF}$ (Week 7), the velocity is 0.802 m/s East, 0.610 m/s North and −0.033 m/s Up: a pedestrian walking at 1.01 m/s on a heading of 52.7°. The values used to simulate the data were 0.80 m/s East, 0.60 m/s North and $d = 45.0$ m/s, so the remaining differences are caused by the Doppler noise.

4.6 Error budget and expected accuracy

Table 9 lists typical one-sigma contributions to the user equivalent range error (UERE) of single-frequency SPP in open-sky conditions. Assuming independent zero-mean errors, they add in root-sum-square (RSS).

Table 9. Typical single-frequency SPP error budget (open sky).

Error source	1σ (m)	Mitigation
Satellite clock (broadcast)	1.5	precise clock products
Broadcast ephemeris	2.0	precise orbit products
Ionosphere (after Klobuchar)	5.0	dual-frequency combination, SBAS or regional models
Troposphere (after model)	0.5	Saastamoinen or similar models
Multipath (open sky)	1.0	antenna siting, weighting, masks; much larger in urban canyons
Receiver noise	0.3	carrier smoothing
UERE (root-sum-square)	5.7	$\sqrt{1.5^2 + 2.0^2 + 5.0^2 + 0.5^2 + 1.0^2 + 0.3^2} = 5.7$ m

The position error is approximately the UERE multiplied by the DOP. With HDOP = 1.5 and VDOP = 2.5, $\sigma_H \approx 1.5 \times 5.7 \approx 8.6$ m and $\sigma_V \approx 2.5 \times 5.7 \approx 14$ m (1σ); the 95 % values are roughly twice as large. In an urban canyon multipath and NLOS reception dominate the budget, the errors are no longer zero-mean, and errors of tens of metres are common. Reducing these errors is the subject of the accuracy challenge.

5 Field Data Collection with a u-blox Receiver

Table 10 lists the software and code. Install u-center, RTKLIB and Python before the session; MATLAB is optional because the CSV files of the example dataset are provided.

Table 10. Software and code used in the laboratory.

Category	Component	Details
Software	u-center (u-blox)	configuration and logging of u-blox receivers, www.u-blox.com
Software	RTKLIB 2.5 (rtklibexplorer)	RTKCONV, RTKPOST and RTKPLOT for conversion and post-processing
Software	Python 3.9 or later	numpy, pandas, matplotlib, pyubx2; optional georinex, pymap3d
Software	MATLAB (optional)	with MatRTKLIB for rinex2csv.m; Mapping Toolbox for plots
Software	Google Earth	visualisation of .kml trajectories
Code	rinex2csv.m	converts RINEX .obs/.nav to the CSV inputs of the SPP code (course GitHub)
Code	pyubx2_csv_converter_gui.py	converts UBX logs to CSV, incl. NAV-HPPOSECEF (course GitHub)
Code	example SPP code (2026/27)	baseline WLS-SPP in Python, provided with the example dataset
Code	tools/check_submission.py	checks the format of your CSV file before upload (Section 9; course GitHub)

The 2026/27 course repository (github.com/weisongwen/AAE4203-S12627, folder Lab1) contains WLSE_spp.py, rinex2csv.m, the pyubx2 converter, MatRTKLIB and tools/check_submission.py, together with the example dataset (Lab1/example) and the benchmark files (Lab1/benchmark).

5.1 Receiver kit

Each group uses a u-blox ZED-F9P receiver, a multi-band antenna and a laptop (Fig. 4). The ZED-F9P tracks GPS, GLONASS, Galileo and BeiDou on two frequency bands simultaneously and outputs raw pseudorange, carrier-phase and Doppler measurements (UBX-RXM-RAWX) together with its own navigation solution.



Fig. 4. The u-blox GNSS receiver kit.

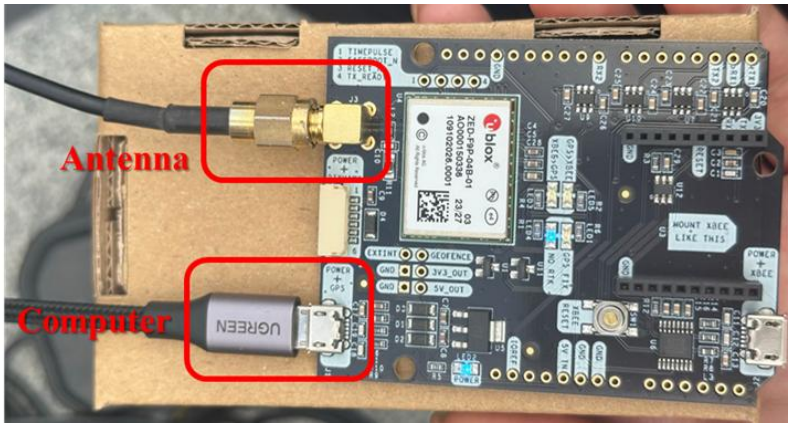


Fig. 5. Connection of the u-blox ZED-F9P board, antenna and laptop.

5.2 Logging raw data with u-center

1. Install u-center from the u-blox website (u-center 1 supports the M8, M9 and F9 generations).
2. Connect the antenna, receiver and laptop. In u-center select the COM port of the receiver and the baud rate (9600 or 115200 bit/s) to establish communication (Fig. 6).
3. Check the data stream: satellites, C/N₀ and position should appear (Fig. 7). If you connect a kit for the first time, enable the raw-measurement messages (Receiver → Debug Message) before logging.
4. Click Record, choose a file name such as G05_owndata_20261030.ubx, and log for 15–20 minutes without moving the antenna (Fig. 8).
5. Click Stop to close the file (Fig. 10). Optionally export the receiver track as KML (Fig. 11).

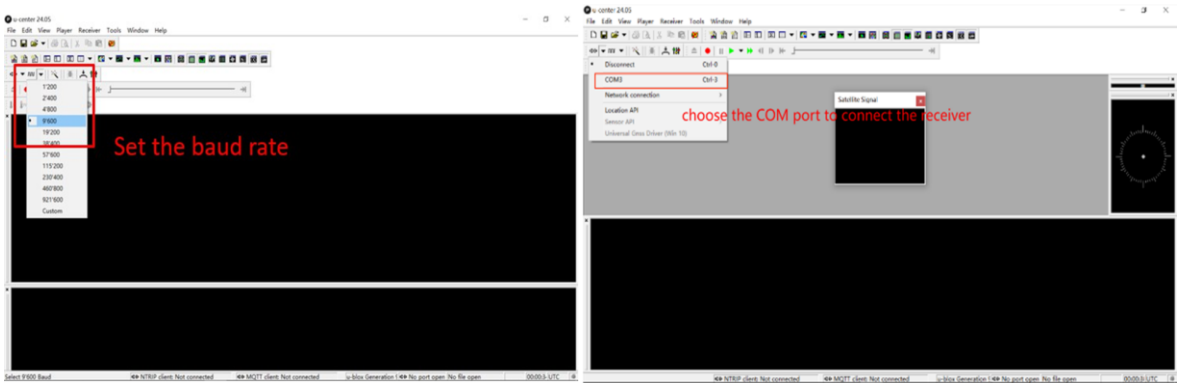


Fig. 6. u-center: connecting the receiver (COM port and baud rate).

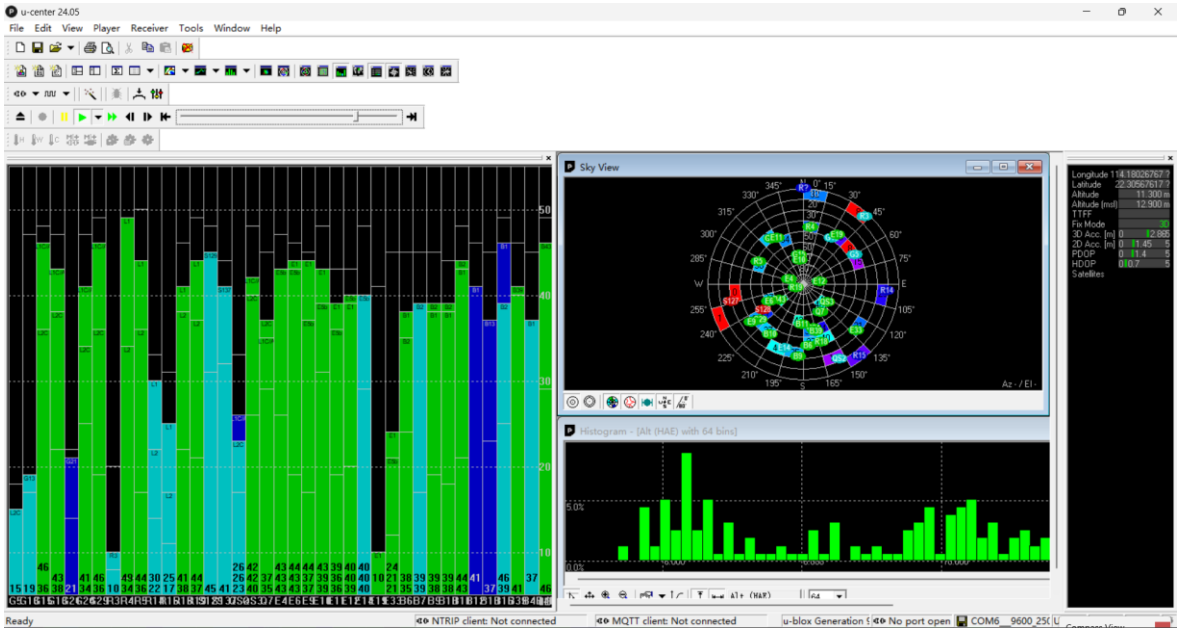


Fig. 7. u-center: satellite view with C/N₀ and sky plot.

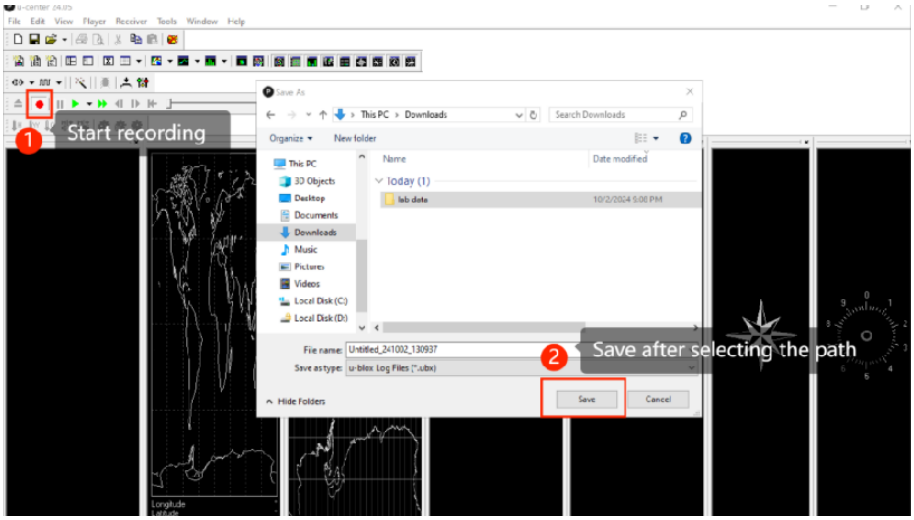


Fig. 8. u-center: starting a recording.

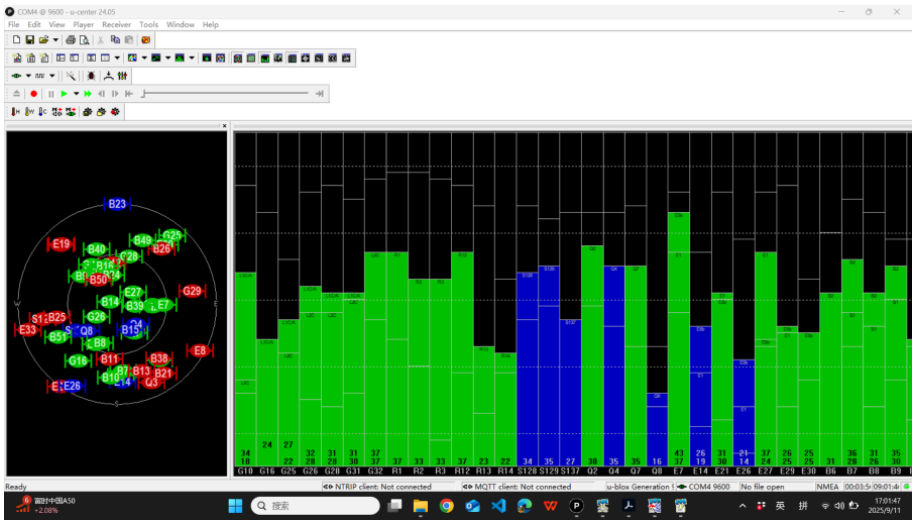


Fig. 9. u-center: data view during recording.

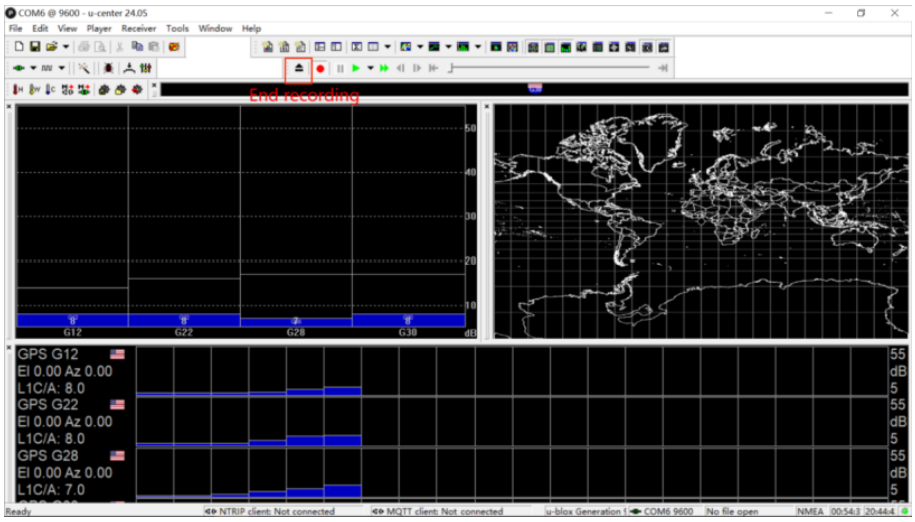


Fig. 10. u-center: stopping the recording.

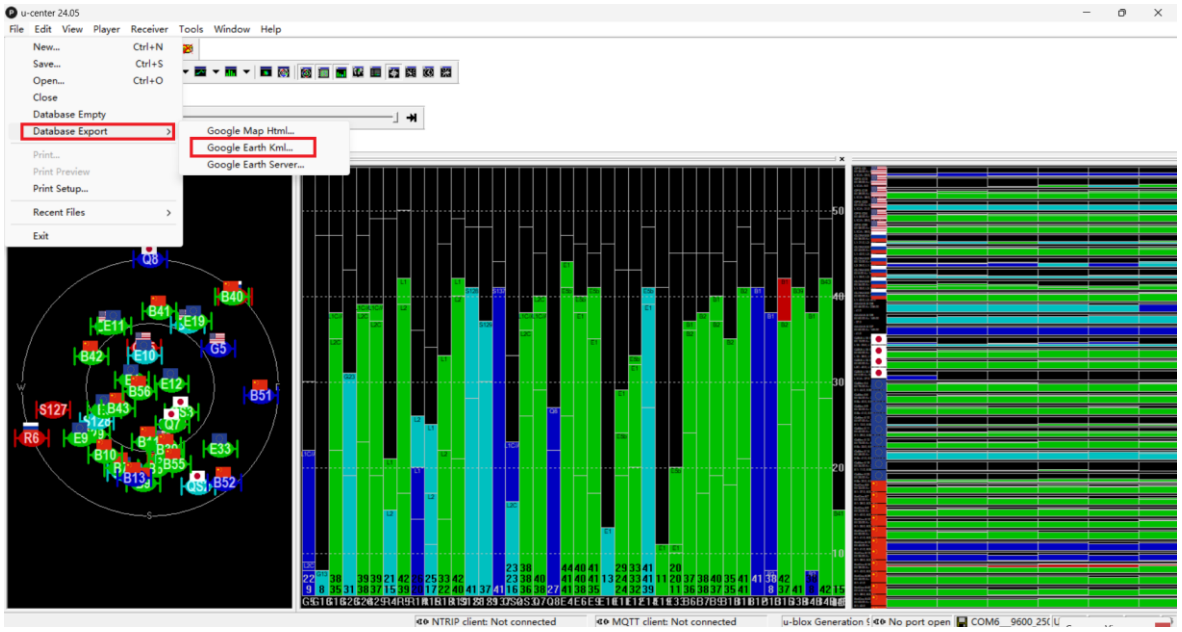


Fig. 11. u-center: exporting the receiver solution to KML.

5.3 Site and procedure

Collect a static dataset in the semi-open area next to Block X (Fig. 12). The site is surrounded by tall buildings, which create a typical urban canyon in which signals are blocked or reflected.

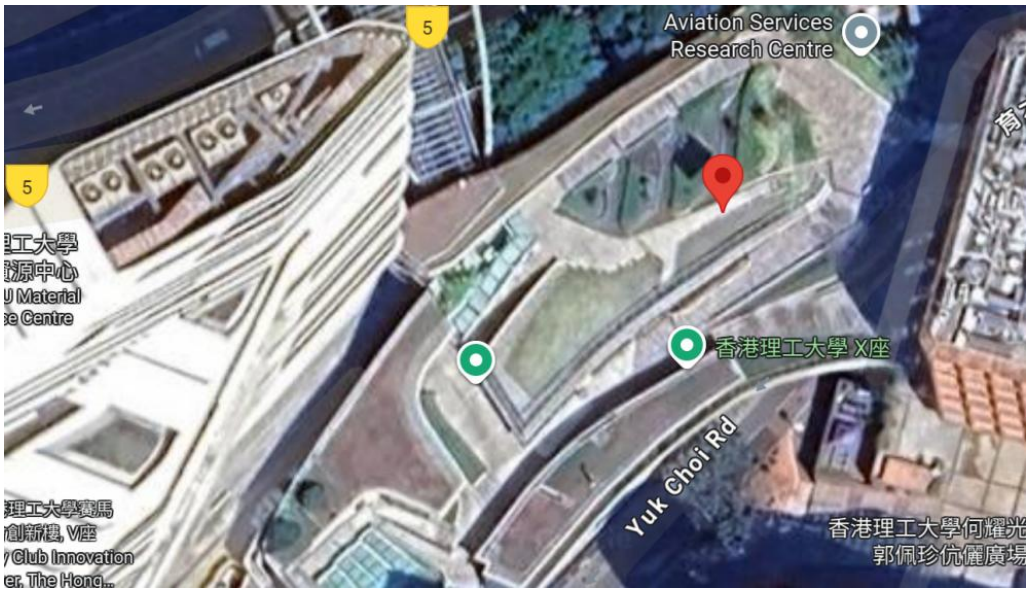


Fig. 12. Data collection site next to Block X, PolyU (red marker).



Fig. 13. Environment of the collection site.

1. Place the antenna at the marked location, on a stable surface and away from your body and laptop.
2. Take photographs of the set-up and of the sky in four directions; note the start and end times and the weather.
3. Log for 15–20 minutes; do not move the antenna while logging.
4. Before you leave the site, copy the UBX file to a second storage location.

6 Processing with RTKLIB

RTKLIB is an open-source package for standard and precise GNSS positioning. Download the rtklibexplorer version 2.5, unzip it and start rtklaunch.exe (Fig. 14).

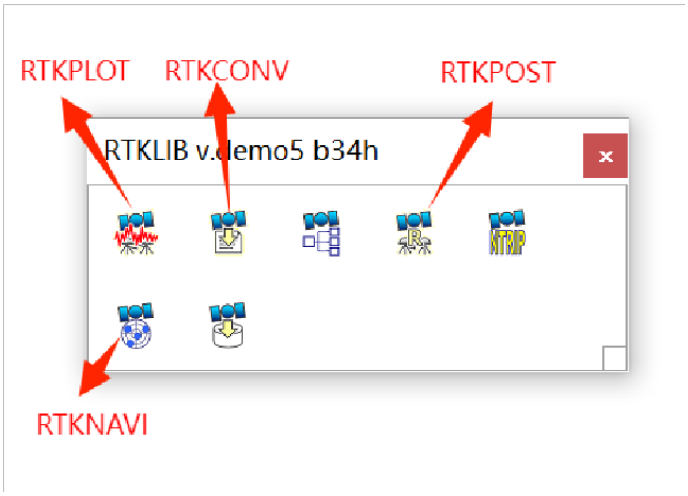


Fig. 14. The RTKLIB launcher.

6.1 Conversion to RINEX with RTKCONV

RINEX (Receiver Independent Exchange Format) is the standard format for raw GNSS data. In RTKCONV select the .ubx file, set the format to u-blox, tick the .obs and .nav outputs, and click Convert (Fig. 15).

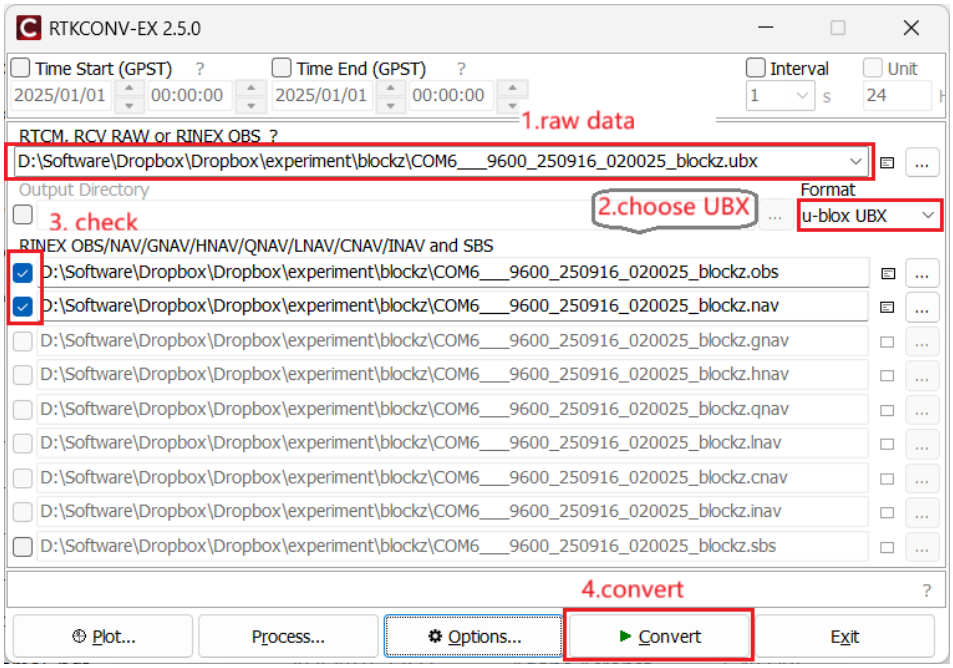


Fig. 15. RTKCONV: conversion of the UBX log to RINEX observation and navigation files.

6.2 Reference-station data from HK SatRef

For differential GNSS (DGNSS) or RTK, download the observations of a nearby reference station from the Hong Kong Satellite Positioning Reference Station Network (SatRef) of the Lands Department (www.geodetic.gov.hk). Read the start and end times of your recording from the header of the .obs file (in GPS time), select a station such as HKSC or HKKT, enter the UTC period, and download the RINEX file (Figs. 16–20).

Step 1. Select the required stations

Hong Kong SatRef Station			
☐**HKKS** Kau Sai Chau	☐**HKKT** Kam Tin	☐**HKLM** Lamma Island	☐**HKLT** Lam Tei
☐**HKMW** Mui Wo	☐**HKNP** Ngong Ping	☐**HKOH** Obelisk Hill	☐**HKPC** Peng Chau
☑**HKSC** Stonecutters Island	☐**HKSL** Siu Lang Shui	☐**HKSS** Shap Sze Heung	☐**HKST** Sha Tin
☐**HKTK** Sha Tau Kok	☐**HKWS** Wong Shek	☐**HKCL** Chek Lap Kok	☐**HKQT** Quarry Bay
☐**T430** Trig 430	☐**HKFN** Fanling	☐**KYC1** Kau Yi Chau	**TCHK*** Tate's Cairn
Macao MoSRef Station			
☐**COAL** Coloane Alto	☐**DSMG** Taipa Grande	☐**FOMO** Monte Fortress	

*Note: 1. Satellite Reference Station summary has been replaced by Satellite Reference Station Coordinates List since 1 Apr, 2016.
2. Data of TCHK is not available

Step 2. Select Data Format

File Length	Interval	Data Format	
		RINEX 2.11	RINEX 3.02
1 Hour	1 Second	**Not Select**	**Selected**
	5 Seconds	**Not Select**	**Not Select**
24 Hours	5 Seconds	**Not Select**	Not Available
	30 Seconds	**Not Select**	**Not Select**

Fig. 16. HK SatRef: selection of the station and the data type.

```
     3.04           OBSERVATION DATA    M: Mixed            RINEX VERSION / TYPE
RTKCONV-EX 2.5.0                       20250916 020447 UTC PGM / RUN BY / DATE
format: u-blox UBX                                         COMMENT
log: D:\Software\Dropbox\Dropbox\experiment\COM6___9600_2509COMMENT
  16_020025_blockz.ubx                                     COMMENT
                                                           MARKER NAME
                                                           MARKER NUMBER
                                                           MARKER TYPE
                                                           OBSERVER / AGENCY
                                                           REC # / TYPE / VERS
                                                           ANT # / TYPE
 -2418220.5428  5385748.0793  2405783.9573                 APPROX POSITION XYZ
        0.0000        0.0000        0.0000                 ANTENNA: DELTA H/E/N
G    8 C1C L1C D1C S1C C2X L2X D2X S2X                     SYS / # / OBS TYPES
R    8 C1C L1C D1C S1C C2C L2C D2C S2C                     SYS / # / OBS TYPES
E    8 C1X L1X D1X S1X C7X L7X D7X S7X                     SYS / # / OBS TYPES
J    8 C1C L1C D1C S1C C2X L2X D2X S2X                     SYS / # / OBS TYPES
S    4 C1C L1C D1C S1C                                     SYS / # / OBS TYPES
C    8 C2I L2I D2I S2I C7I L7I D7I S7I                     SYS / # / OBS TYPES
  2025    09    16    02    00    54.0060000     GPS       TIME OF FIRST OBS      Start Time
  2025    09    16    02    03    59.0060000     GPS       TIME OF LAST OBS       End Time
G L1C                                                      SYS / PHASE SHIFT
G L2X  0.00000                                             SYS / PHASE SHIFT
R L1C                                                      SYS / PHASE SHIFT
R L2C                                                      SYS / PHASE SHIFT
E L1X  0.00000                                             SYS / PHASE SHIFT
E L7X  0.00000                                             SYS / PHASE SHIFT
J L1C                                                      SYS / PHASE SHIFT
J L2X  0.00000                                             SYS / PHASE SHIFT
S L1C                                                      SYS / PHASE SHIFT
```

Fig. 17. Start and end times in the header of the observation file.

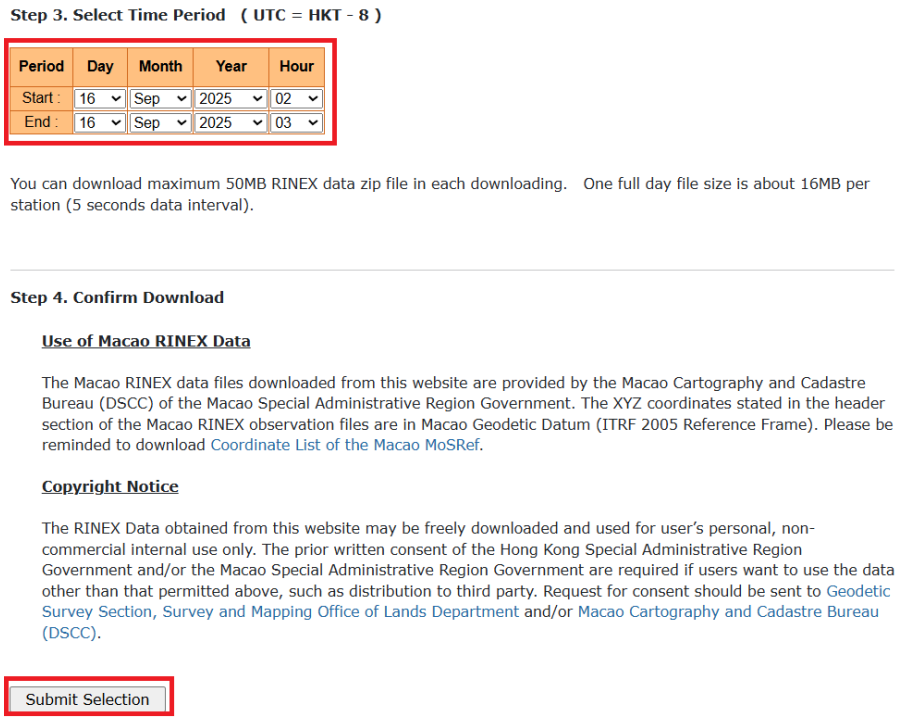


Fig. 18. HK SatRef: entering the time period.



Fig. 19. HK SatRef: output folder of the request.



Fig. 20. Downloaded reference-station files.

6.3 Post-processing with RTKPOST

In RTKPOST select the rover .obs file, the .nav file and, for DGNSS, the base-station observation file. Choose the positioning mode Single for SPP (Fig. 21) or DGPS/Kinematic for differential solutions (Fig.

22), and click Execute. Click Plot to open RTKPLOT; for a static dataset the ground track should be a compact cluster (Figs. 23–24). Finally convert the .pos file to KML (Tools → Convert to KML) and import it into Google Earth (Figs. 25–27).

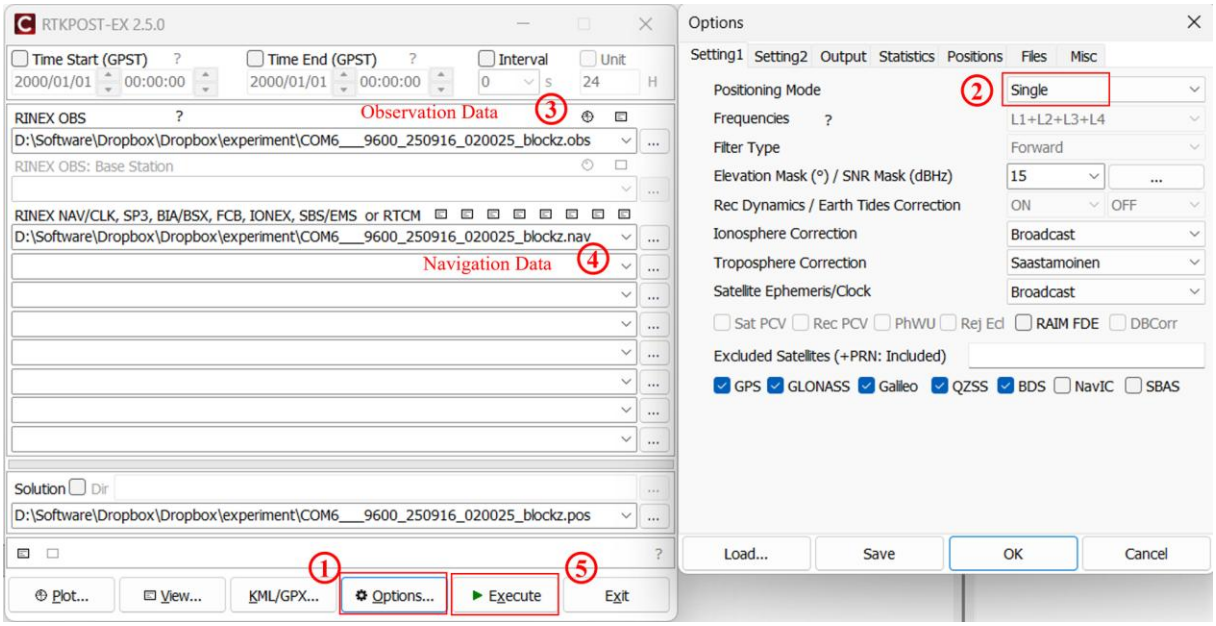


Fig. 21. RTKPOST settings for an SPP solution.

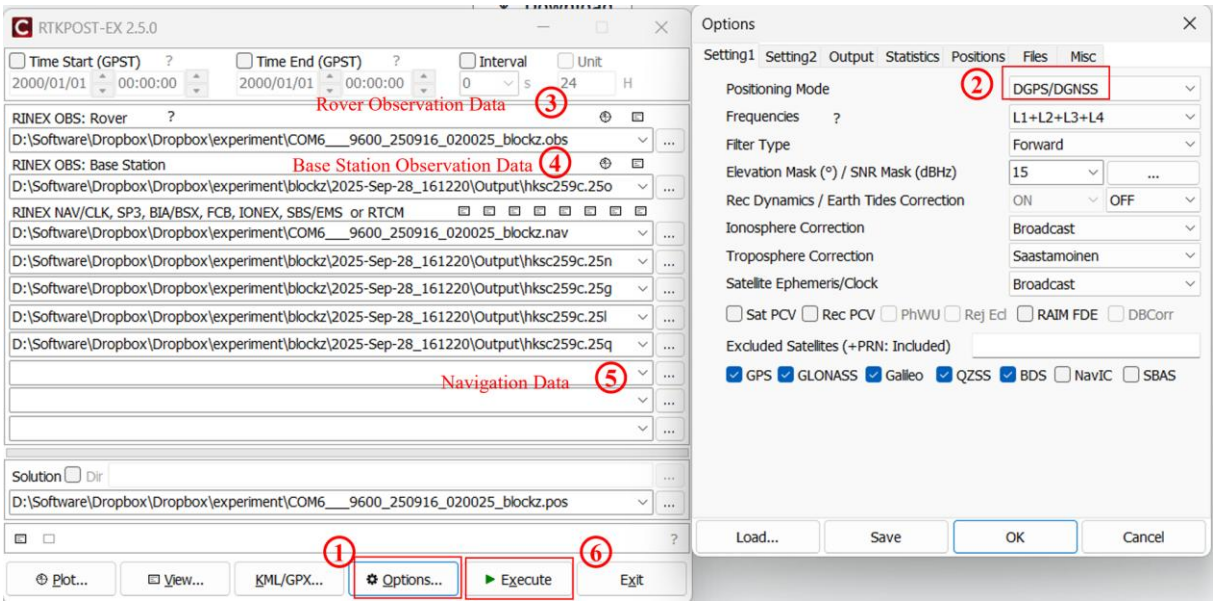


Fig. 22. RTKPOST settings for a differential solution with a SatRef base station.

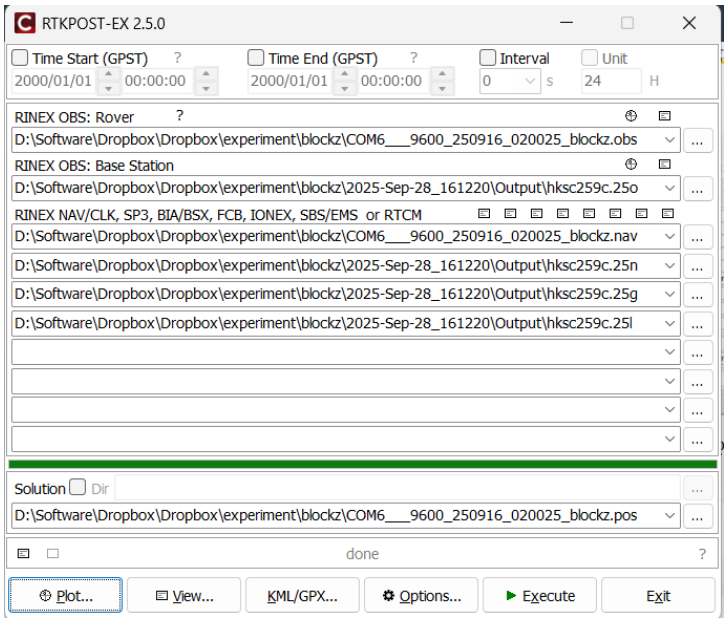


Fig. 23. RTKPOST: the Plot button opens RTKPLOT.

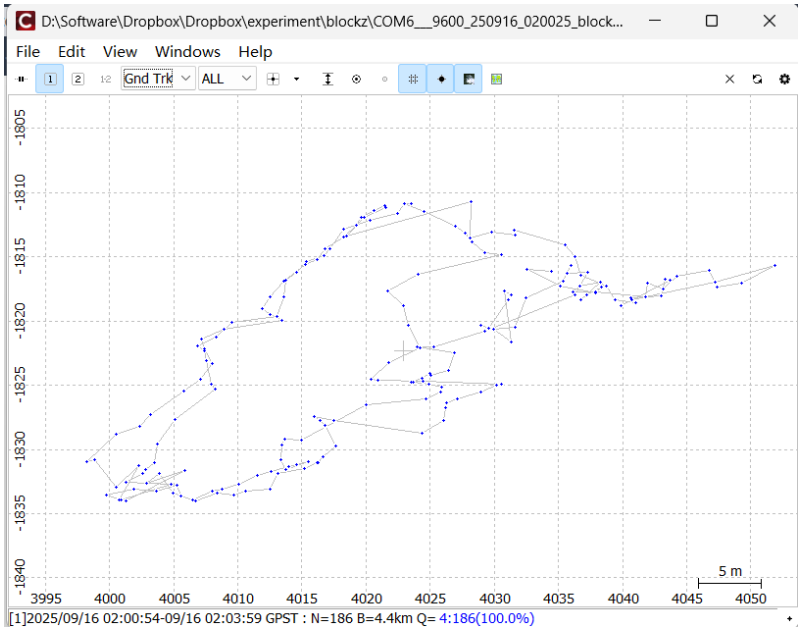


Fig. 24. RTKPLOT: ground track of a static dataset.

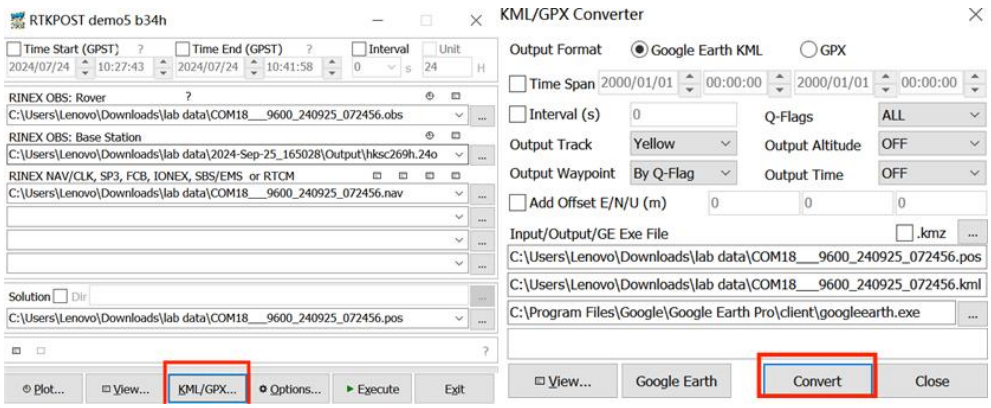


Fig. 25. Conversion of the .pos solution to KML.

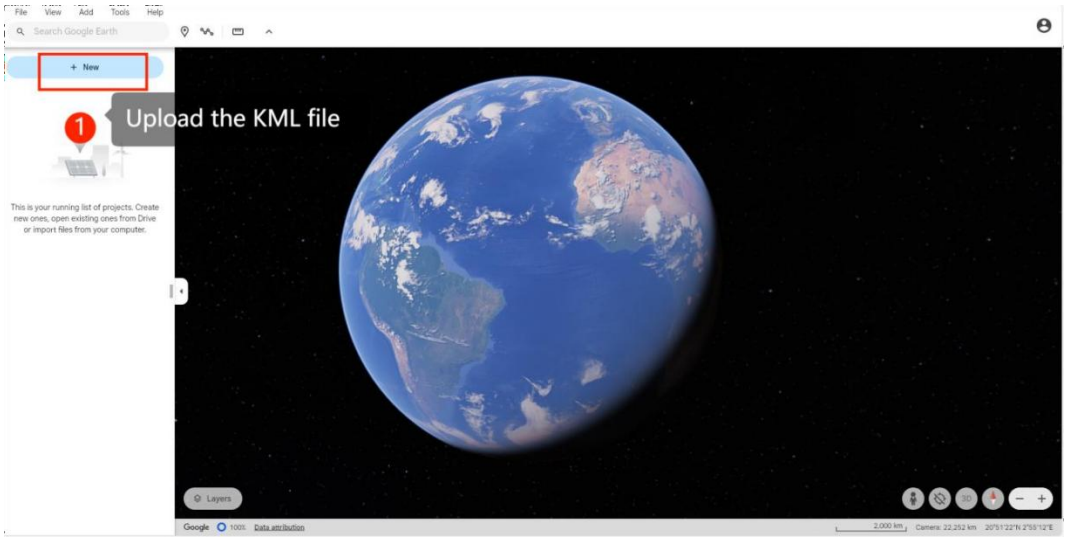


Fig. 26. Importing the KML file into Google Earth.

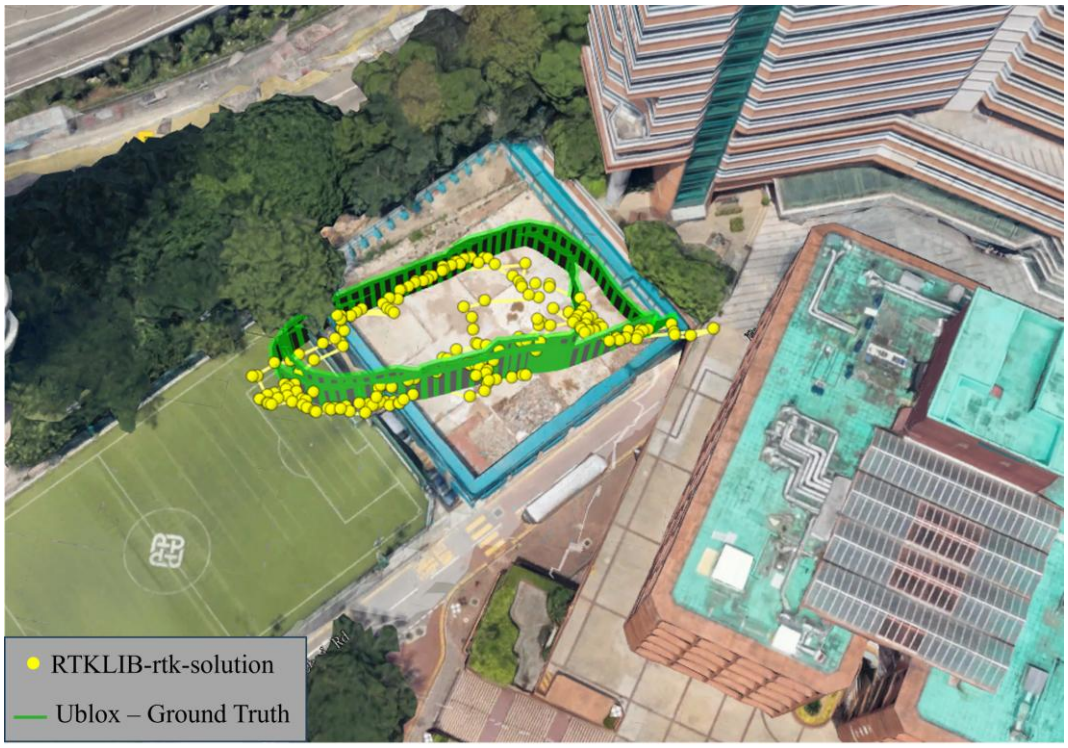


Fig. 27. RTKLIB solution (yellow) and receiver solution (green, labelled "Ground Truth" in the 2025/26 screenshot) displayed in Google Earth.

7 Conversion of Raw Data to CSV

7.1 RINEX to CSV with rinex2csv.m

The MATLAB script rinex2csv.m reads the RINEX files with MatRTKLIB, computes satellite positions and clock offsets from the broadcast ephemeris, evaluates the ionospheric and tropospheric models, and writes one CSV file per quantity: pseudoranges_meas.csv, satellite_clock_bias.csv, satellite_positions.csv, ionospheric_delay.csv and tropospheric_delay.csv. These files are the inputs of the example Python code. The same script plots the number of satellites, C/N₀ and the sky plot, which you need for the raw-data analysis (Figs. 28–30).

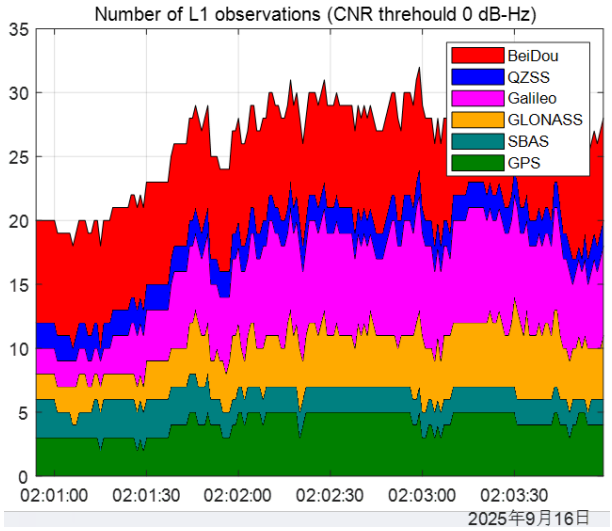


Fig. 28. Number of L1 observations per constellation over time (example dataset).

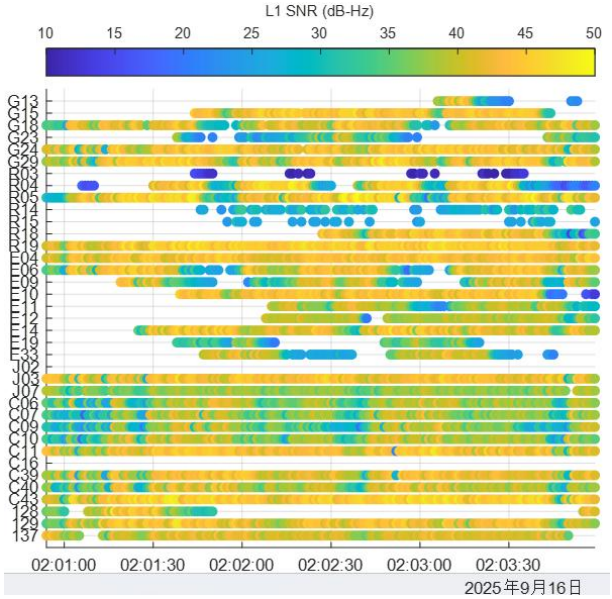


Fig. 29. L1 C/N₀ of all tracked satellites over time (example dataset).

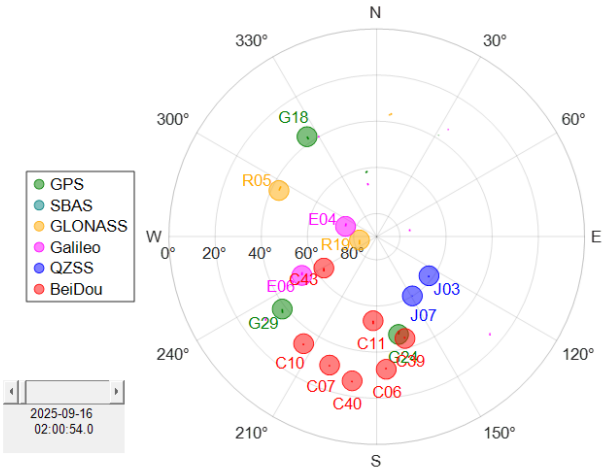


Fig. 30. Sky plot of the example dataset.

Signals above about 40 dB-Hz normally indicate direct line-of-sight reception, whereas values below about 30 dB-Hz at moderate elevation often indicate diffraction, multipath or NLOS reception. Use these thresholds as a starting point for the masks and weights in Section 8.

7.2 Receiver solution from UBX with pyubx2

The GUI script pyubx2_csv_converter_gui.py (install pyubx2 with pip install pyubx2) converts a UBX log to one CSV file per message type. The file NAV-HPPOSECEF.csv contains the receiver's own high-precision ECEF solution; the epoch is given in milliseconds of GPS week (iTOW) and the coordinates in centimetres with a 0.1 mm high-precision part.

Receiver solution versus ground truth

- Without RTK corrections, NAV-HPPOSECEF is the receiver's standalone real-time solution. In an urban canyon it can also be wrong by several metres; it is a useful reference, but not a ground truth.
- For your own data, compare your SPP with RTKLIB and with the receiver solution and discuss the differences; do not call any of them the truth.
- Accuracy against a true trajectory is measured only on the benchmark dataset.

8 SPP Implementation and Accuracy Improvement

8.1 Baseline implementation

The example code implements the algorithm of Section 4 for one epoch at a time: (1) read the CSV inputs; (2) correct the pseudoranges for the satellite clock, ionosphere and troposphere; (3) iterate Eq. (5) until convergence; (4) convert the result to latitude, longitude and height and to ENU errors with respect to a reference; (5) write the trajectory and statistics. Run it first on the example dataset and confirm that you reproduce the results in Section 8.6.

8.2 Ideas for improving accuracy

Table 11 lists techniques that are known to reduce SPP errors in urban areas. Change one thing at a time, measure its effect, and keep a table of the results for the report (an ablation study).

Table 11. Techniques for improving SPP accuracy in urban environments.

Technique	Idea	Comment
Multi-constellation	add BDS and Galileo (one clock bias per system, Eq. (6))	more satellites and better geometry; usually the largest single gain
Weighting	$w_i = \sin^2\theta_i$, or C/N₀-dependent variances, e.g. $\sigma_i^2 = a^2 + b^2/\sin^2\theta_i$	down-weights low and weak signals
Masks	elevation mask (e.g. 10–15°) and C/N₀ mask (e.g. 25–30 dB-Hz)	removes the worst signals but weakens the geometry if too strict
Outlier rejection	residual test: remove the satellite with the largest normalised residual, re-solve, repeat	fault detection and exclusion, as in receiver autonomous integrity monitoring (RAIM)
Atmospheric models	check the Klobuchar and Saastamoinen corrections; use dual-frequency data if available	removes biases of several metres
Time-domain filtering	carrier smoothing (Hatch filter), or a Kalman filter with Doppler velocity	reduces noise; check that it does not add lag in dynamic segments

8.3 Working with AI tools

AI assistants are most useful when the request is precise and the result is checked against known data. The prompts below are examples; adapt them to your files and code.

Listing 1. Example prompts for AI-assisted development.

```
Prompt 1  (baseline)
I have CSV files pseudoranges_meas.csv, satellite_positions.csv, satellite_clock_bias.csv,
ionospheric_delay.csv and tropospheric_delay.csv (one row per epoch, columns described below).
Write a Python function that computes the weighted least-squares SPP solution for each epoch,
with elevation-dependent weights sin^2(el), Gauss-Newton iterations stopped at 1e-4 m, and
output gps_week, gps_tow_s, x_ecef_m, y_ecef_m, z_ecef_m. Explain every step.

Prompt 2  (improvement)
Extend this SPP code to GPS + BeiDou + Galileo with one receiver clock bias per system, and add
a residual-based outlier rejection loop. Keep the original version for comparison.

Prompt 3  (verification)
Write a script that compares my trajectory with the reference solution of the example dataset
and prints the East/North/Up RMSE, the 2D and 3D RMSE and the 95th percentile.
```

8.4 Verification on the example dataset

- Run every new version on the example dataset first and compare with the provided results; investigate any change larger than a few centimetres in the baseline.
- Plot the residuals and the number of satellites per epoch; a sudden jump usually indicates a bug or a bad satellite.
- Check units (metres versus centimetres, seconds versus milliseconds) and time systems (GPST, BDT, UTC).
- Only then process the benchmark and record its metrics for every version you submit.

8.5 Validating your own data without ground truth

Your own data have no ground truth, so their true error cannot be computed. Validation instead asks two questions: is the processing chain working, and how does the solution behave? Three complementary checks are required for the report (Table 12).

Table 12. Validation checks for the self-collected data.

Check	What to compute	What it reveals
Comparison of solutions	differences between your SPP, RTKPOST SPP and the receiver solution, in E/N/U	processing errors (units, time tags, corrections)
Consistency of a static solution	mean, standard deviation and 95 % horizontal radius about the mean; East–North scatter plot	noise level and slow multipath drifts
Input quality	satellites per epoch and per system, C/N₀, elevation, DOP, post-fit residuals	causes of large errors at particular epochs

For the comparison of two solutions a and b, rotate the difference at every common epoch k into the local ENU frame and summarise it by its mean and standard deviation for each component $c \in \{E, N, U\}$:

$$d_k = [d_{E,k},\, d_{N,k},\, d_{U,k}]^T = R(\varphi, \lambda)\,(\hat{p}_k^a - \hat{p}_k^b), \tag{15}$$

$$\overline{d}_c = \frac{1}{K}\sum_{k=1}^{K} d_{c,k}\,, \qquad s_c = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}\left(d_{c,k} - \overline{d}_c\right)^2}, \tag{16}$$

where K is the number of common epochs and (φ, λ) is the mean position of solution b.

Small differences show that two processing chains agree; they do not show that either is accurate. In a street canyon all three solutions use the same reflected signals and can share the same multipath error, which is precisely why the benchmark is evaluated against an independent ground truth.

8.6 Example results on the Block X dataset

Figs. 31–35 show the results of the reference WLS-SPP implementation on the example dataset (186 epochs), compared with the receiver reference solution. The large 2D RMSE of about 38 m illustrates how strongly the urban environment near Block X degrades GPS-only SPP, and leaves ample room for improvement. These plots were produced in 2025/26, when the receiver solution was labelled “Ground Truth”; read that label as “receiver reference solution” (Section 7.2).

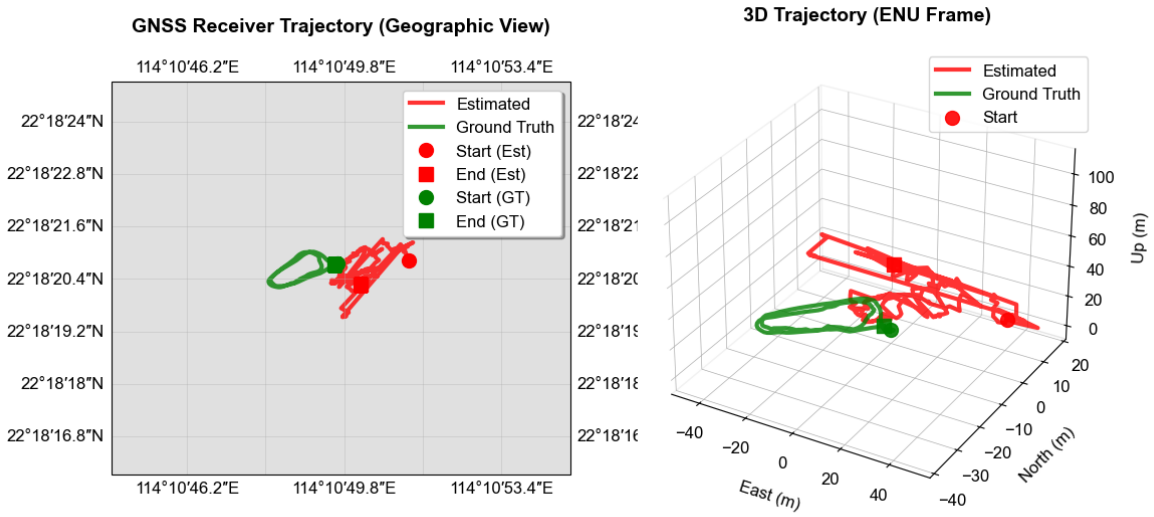


Fig. 31. Example dataset: estimated trajectory in geographic coordinates and in the ENU frame.

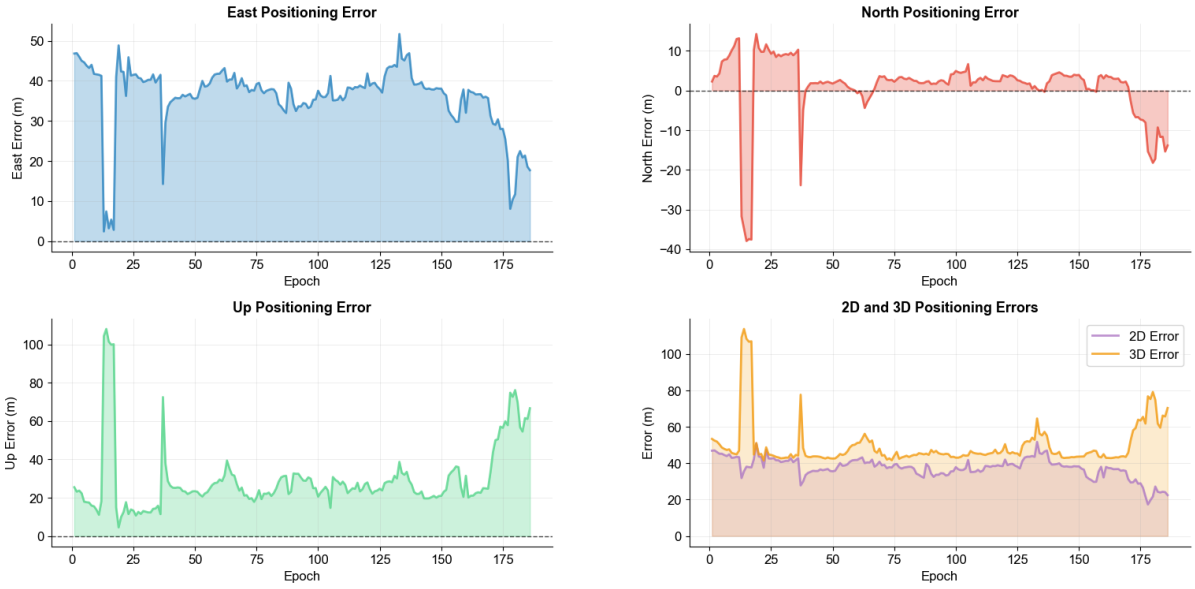


Fig. 32. Example dataset: East, North and Up errors and 2D/3D errors over time.

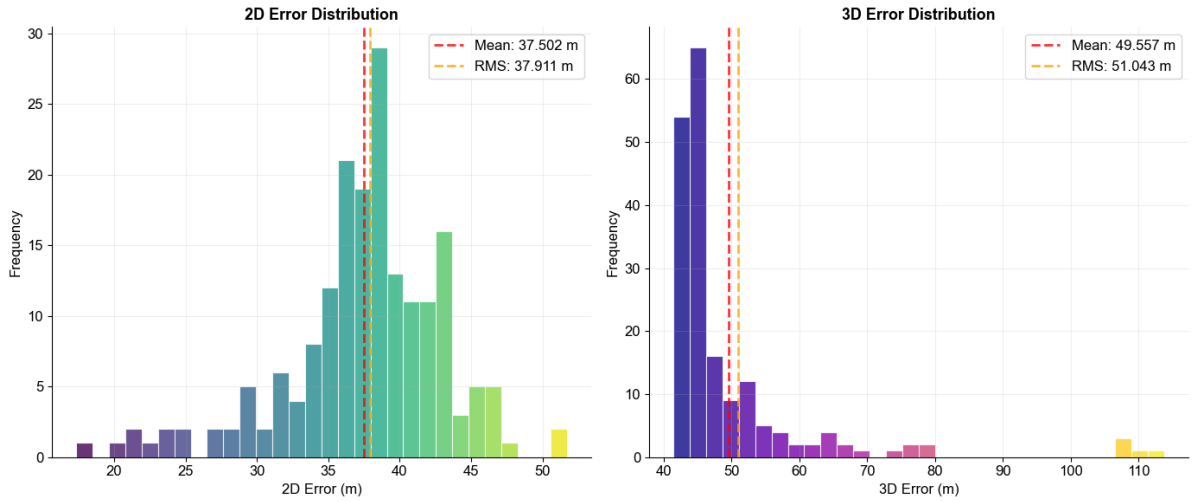


Fig. 33. Example dataset: histograms of the 2D and 3D errors.

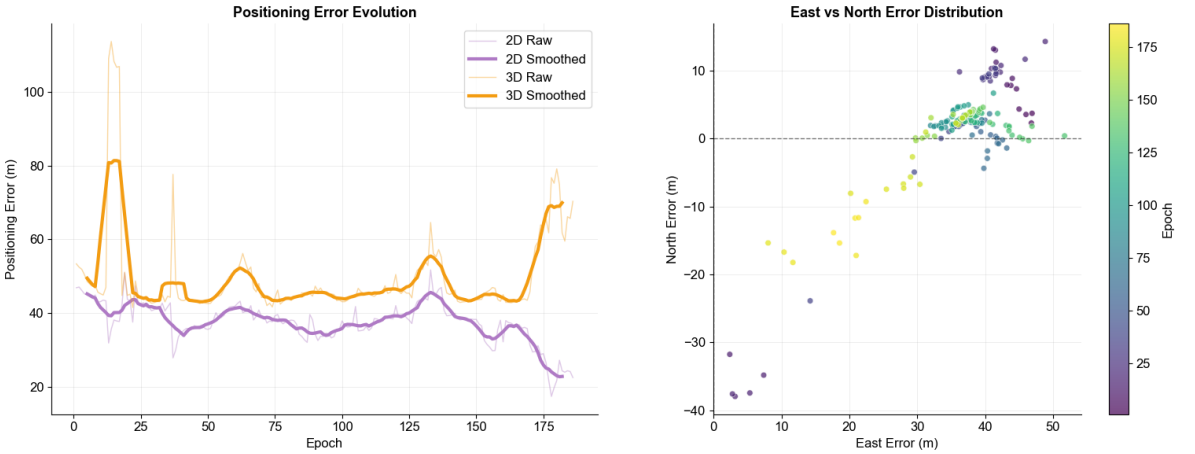


Fig. 34. Example dataset: error evolution and East-versus-North scatter.

```
================================================================
        GNSS SPP POSITIONING RESULTS SUMMARY
================================================================
Total Epochs Processed: 186
================================================================
        POSITIONING ACCURACY STATISTICS
================================================================
Metric              2D Error (m)    3D Error (m)
----------------------------------------------------------------
Mean                37.502          49.557
RMS                 37.911          51.043
Standard Deviation  5.553           12.227
Maximum             51.699          113.729
Minimum             17.350          41.472
95th Percentile     45.208          73.716
================================================================
        COORDINATE RANGES
================================================================
Latitude Range:  22.305424°  to 22.305920°
Longitude Range: 114.180416°  to 114.180938°
Altitude Range:  10.550 m to 114.610 m
================================================================
```

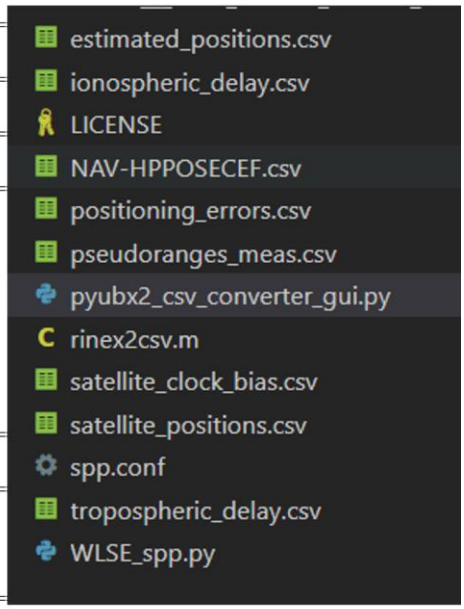


Fig. 35. Example dataset: summary of the positioning statistics and output files.

Table 13. Accuracy statistics of the reference implementation on the example dataset.

Metric	2D error (m)	3D error (m)
Mean	37.502	49.557
RMS	37.911	51.043
Standard deviation	5.553	12.227
Maximum	51.699	113.729
Minimum	17.350	41.472
95th percentile	45.208	73.716

9 Accuracy Challenge: Submission and Leaderboard

Process the benchmark dataset (RINEX observation and navigation files from UrbanNav [4]) with your best algorithm and submit one position per epoch. The reference trajectory is kept by the teaching team for evaluation; the leaderboard and group results pages provide feedback on your accuracy, which is why each version should first be checked on the example dataset.

9.1 CSV file format

Submit a comma-separated text file with exactly the five columns of Table 14 and a header line. Use one row per epoch of the benchmark observation file, in time order; the epochs are matched to the ground truth by GPS week and time of week (to 0.1 s). Epochs without a solution may be omitted, but they reduce the availability.

Table 14. Columns of the submission file.

Column	Meaning	Unit / format	Example
gps_week	GPS week number	integer	2158
gps_tow_s	GPS time of week of the epoch	s, at least 1 decimal	455346.0
x_ecef_m	Estimated receiver position, ECEF X	m, 3 decimals	−2418202.586
y_ecef_m	Estimated receiver position, ECEF Y	m, 3 decimals	5385847.524
z_ecef_m	Estimated receiver position, ECEF Z	m, 3 decimals	2405648.799

Listing 2. First lines of a valid file G05_benchmark.csv (illustrative values).

```
gps_week,gps_tow_s,x_ecef_m,y_ecef_m,z_ecef_m
2158,455346.0,-2418202.586,5385847.524,2405648.799
2158,455347.0,-2418202.174,5385847.238,2405648.952
2158,455348.0,-2418201.791,5385846.993,2405649.187
...
```

- File names: GXX_benchmark.csv for the benchmark and GXX_owndata.csv for your own data, where XX is your two-digit group number (G01–G20).
- Positions are ECEF coordinates in metres (WGS-84) with three decimals; no latitude/longitude, no extra columns.
- Run python tools/check_submission.py GXX_benchmark.csv and fix all errors before uploading.

9.2 Upload

The TA posts the link to a shared Google Drive folder on Blackboard; it contains one sub-folder per group. Upload both CSV files to your group's sub-folder and replace the file when you have a better solution; only the latest file named GXX_benchmark.csv is evaluated. The platform automatically evaluates the files and updates the live leaderboard. The TA announces the leaderboard link during the session and shows a preliminary ranking at about 2:50. The final leaderboard uses the files present at the submission deadline and is published on Blackboard. The platform source code is available at https://github.com/PolyU-TASLAB/AAE4203_ASN_2026.

9.3 Evaluation metrics

For every matched epoch k, the position error is rotated to the local ENU frame at the ground-truth position p_k^{gt}:

$$[e_{E,k},\, e_{N,k},\, e_{U,k}]^T = R(\varphi_k, \lambda_k)\,(\hat{p}_k - p_k^{\mathrm{gt}}), \tag{17}$$

and with N matched epochs out of N_{gt} ground-truth epochs the metrics are

$$\mathrm{RMSE}_{2D} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(e_{E,k}^2 + e_{N,k}^2\right)}, \qquad \mathrm{RMSE}_{3D} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(e_{E,k}^2 + e_{N,k}^2 + e_{U,k}^2\right)}, \tag{18}$$

$$A = 100\,\frac{N}{N_{\mathrm{gt}}}\ \%, \qquad P_{95} = 95\text{th percentile of } \sqrt{e_{E,k}^2 + e_{N,k}^2}. \tag{19}$$

9.4 Ranking rules

- Only submissions with availability $A \geq 90$ % are ranked; the others are listed as unranked.
- Ranked submissions are ordered by RMSE_{2D} (smaller is better); ties are broken by RMSE_{3D}.
- The leaderboard also reports P_{95} and A for information.
- The repository must reproduce the submitted file; the teaching team may re-run the code of any group.
- How the rank contributes to the laboratory mark is announced by the lecturer on Blackboard.

10 Checklist and Concluding Remarks

Table 15. Checklist for each group.

Before leaving the field	Before the end of the session	Before the deadline
UBX file saved and backed up	example results reproduced	final CSV files uploaded
site photos and sky photos	own data converted and validated	report (PDF) submitted
start/end time, weather	validation plots saved	repository reproduces both CSV files
kit returned complete	benchmark CSV checked and uploaded	AI-use statement included

The laboratory connects the theory of Weeks 4–7 to real measurements: the same least-squares equations that solve a five-satellite textbook example must now cope with blocked and reflected signals in a real city. A good report shows not only the final number on the leaderboard but also what each step contributed and why. Natural extensions are 3D-mapping-aided GNSS, NLOS classification with machine learning, and the RTK techniques of Week 10.

References

[1] T. Takasu and A. Yasuda, “Development of the low-cost RTK-GPS receiver with an open source program package RTKLIB,” in Proc. Int. Symp. GPS/GNSS, Jeju, Korea, 2009.

[2] T. Takasu, RTKLIB ver. 2.4.2 Manual, 2013. [Online]. Available: https://www.rtklib.com; rtklibexplorer version 2.5: https://github.com/rtklibexplorer/RTKLIB

[3] u-blox, ZED-F9P Integration Manual and u-center User Guide, u-blox AG, Thalwil, Switzerland.

[4] L.-T. Hsu, N. Kubo, W. Wen, W. Chen, Z. Liu, T. Suzuki and J. Meguro, “UrbanNav: An open-sourced multisensory dataset for benchmarking positioning algorithms designed for urban areas,” in Proc. ION GNSS+ 2021, pp. 226–256.

[5] E. D. Kaplan and C. J. Hegarty, Eds., Understanding GPS/GNSS: Principles and Applications, 3rd ed. Norwood, MA, USA: Artech House, 2017.

[6] P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed. Lincoln, MA, USA: Ganga-Jamuna Press, 2011.

[7] J. Sanz Subirana, J. M. Juan Zornoza and M. Hernández-Pajares, GNSS Data Processing, Vol. I: Fundamentals and Algorithms, ESA TM-23/1. Noordwijk, The Netherlands: ESA Communications, 2013.

[8] T. Suzuki, MatRTKLIB: MATLAB toolbox for GNSS analysis using RTKLIB. [Online]. Available: https://github.com/taroz/MatRTKLIB

[9] S. Bennett, pyubx2: Python library for parsing and generating u-blox UBX messages. [Online]. Available: https://github.com/semuconsulting/pyubx2

[10] Lands Department, HKSAR, Hong Kong Satellite Positioning Reference Station Network (SatRef). [Online]. Available: https://www.geodetic.gov.hk